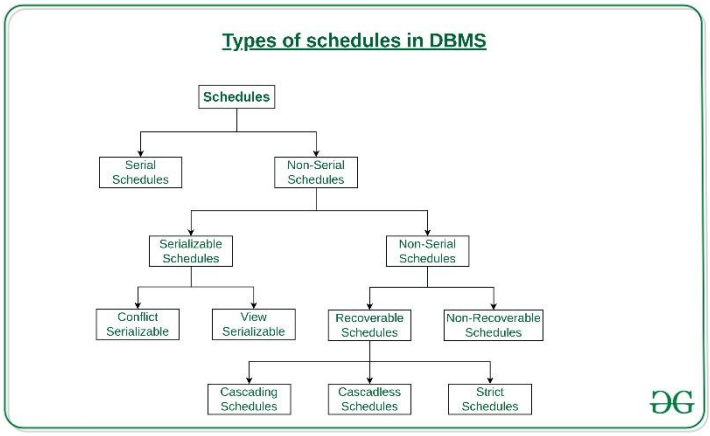
Transaction Schedule and It's Type

Definition of Schedule

A **schedule** is a **sequence of instructions (operations)** from one or more transactions that preserves the **order of operations within each transaction**.

Each transaction consists of a set of operations such as:

- **Read(X)** → Reading data item X
- **Write(X)** → Writing data item X
- **Commit** → Saving the transaction
- **Rollback** → Undoing the transaction



Transaction Schedule and Its Types

1. What is a Transaction Schedule?

A **schedule** in DBMS is the **order of execution** of operations (Read/Write) from multiple transactions.

When multiple transactions run **concurrently**, their instructions may interleave.
This interleaving of operations, while following each transaction's internal order, is called a **Transaction Schedule**.

Simple Definition:

A schedule is the sequence in which multiple transactions' read/write operations are executed.

---

2. Why Schedules Are Needed?

Schedules help the DBMS:

- Maintain **correctness** during concurrency

- Avoid **conflicts**
- Improve **performance**
- Prevent issues like lost updates, dirty reads, and inconsistencies

---

3. Types of Schedules

Schedules are mainly classified as:

1. **Serial Schedule**
2. **Non-Serial Schedule**
3. **Serializable Schedule**
    - Conflict Serializable
    - View Serializable

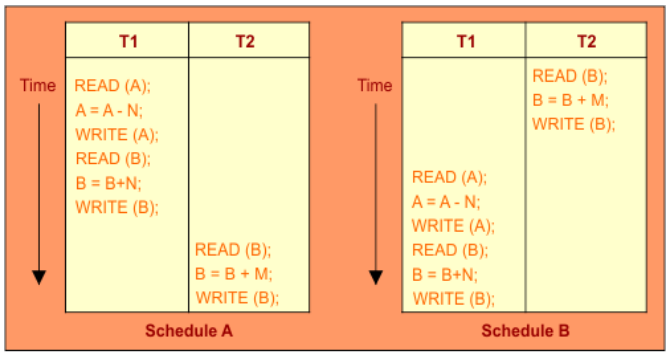
Let's explain each with examples.

3.1 Serial Schedule

Definition

A Schedule in which **transactions execute one after another**, with **no interleaving** of operations.

Only one transaction runs at a time.



Schedules Both A and B are Serial

Example

Transactions:

- **T1:** Read(A), Write(A)
- **T2:** Read(B), Write(B)

Serial Schedule Example:

Schedule S1:

T1: R(A)

T1: W(A)

T2: R(B)

T2: W(B)

Here, **T1 completes first**, then **T2 starts**.
This is always **consistent** but slower.

Advantages:

- Always **consistent and safe**.
- **No conflicts** (no data corruption).

Disadvantage:

- **Slow performance** — no parallelism.

3.2 Non-Serial Schedule

Definition

A **non-serial schedule** allows **interleaving** of operations from multiple transactions

Advantage

Faster performance due to concurrency.

Disadvantage

May cause **inconsistency** if not properly controlled.

Example

Schedule S2:

T1: R(A)

T2: R(B)

T1: W(A)

T2: W(B)

Here, T1 and T2 execute in a mixed order.
Faster, but **may lead to conflicts**.

(c)

T1	T2
read(A); A := A – N;	
	read(A); A := A + M;
write(A); read(B);	
	write(A);
B := B + N; write(B);	

Time

Schedule C

3.3 Serializable Schedules

Definition

A non-serial schedule is **serializable** if its outcome is **equivalent to some serial schedule**.

There are two types:

1. **Conflict Serializable**
2. **View Serializable**

3.3.1 Conflict Serializable Schedule

Definition

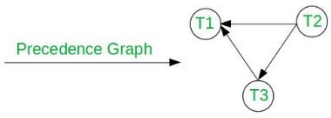
A schedule is conflict-serializable if it can be **converted to a serial schedule** by swapping non-conflicting operations.

Two operations conflict when all are true:

1. Belong to **different transactions**
2. Operate on **same data item**
3. At least one is **Write**

T1	T2	T3
	R(A)	
	W(A)	
		R(C)
	W(B)	
		W(A)
		W(C)
R(A)		
R(B)		
W(A)		
W(B)		

Schedule S



Example

Transactions:

- T1: R(A), W(A)
- T2: R(A), W(A)

Schedule S3:

T1: R(A)

T2: R(A)

T1: W(A)

T2: W(A)

This looks interleaved but can be rearranged to:

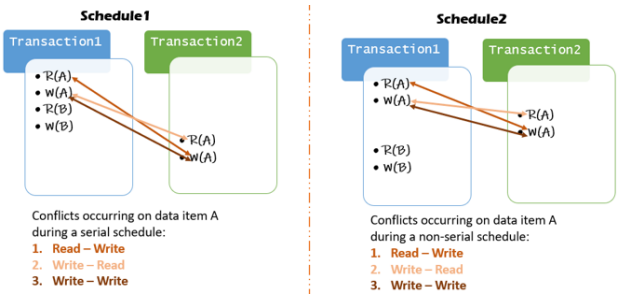
T1: R(A)

T1: W(A)

T2: R(A)

T2: W(A)

So S3 is **conflict serializable**.



3.3.2 View Serializable Schedule

Definition

A schedule is view-serializable if it produces the **same final result** and **same reads** as some serial schedule.

Three conditions must match the serial schedule:

1. **Initial Read** (who reads first)
2. **Intermediate Read** (who reads whose value)
3. **Final Write** (who writes last)

Example

Serial Order Equivalent → T1 → T2

Schedule S4:

T1: R(A)

T2: R(A)

T2: W(A)

Here:

- Both read same initial value
- Final write by T2
- Matches serial behavior of T1 → T2

Thus, S4 is **view serializable** though not conflict serializable.